Lab 4: Abstract classes and interfaces

Dr. Paweł Głąba

April 8, 2026

Deadline: 22.04.2026

Basic Problem

This project is based on a core model: the **Smartwatch**. Every smartwatch provides essential features such as displaying notifications, tracking activity, and connecting wirelessly to other devices. Different manufacturers offer different payment and charging solutions, so the system should be designed using abstraction and interfaces.
Implement an abstract class `Smartwatch` with the following fields:

- serial number – a unique device identifier stored as a `String`
- paired phone number as a 9-digit `String`
- activity history as a list of `String` values
- battery level (from 0 to 100%, defaults to 60% at startup)
- Bluetooth connection status (`false` at the start)
- WiFi connection status (`false` at the start)
- notification history as a list of `String` values
- watch producer represented as an enum `WatchProducer`

Implement a **static factory method** to validate input values before creating an object. If any value is invalid, the method should throw an IllegalArgumentException with an appropriate error message
Create two subclasses:

- `AndroidWatch`
- `AppleWatch`

When creating an `AppleWatch` object, the producer must be `APPLE`. When creating an `AndroidWatch` object, the producer must be one of the Android-based manufacturers defined in the `WatchProducer` enum. If the producer does not match the watch type, throw an `IllegalArgumentException`. Implement the following interfaces:

- `Trackable` – handles activity tracking.
  - `startWorkout(String workoutType)`
  - `stopWorkout()`
- `Connectable` – manages wireless connections. Provide methods to connect and disconnect from WiFi and Bluetooth. This interface should be implemented in the `Smartwatch` class.
- `Payable` – handles contactless payments. Since Android and Apple devices use different payment systems, this interface should be implemented in child classes.
- `Chargeable` – handles charging. Since different devices use different charging solutions, this interface should be implemented in child classes.
- `Notifiable` – handles notifications.
  - `receiveNotification(String message)`
  - `showNotificationHistory()`

Override the `toString()` method to display the current watch status. Below is an example output:

```
Watch Info:
Producer: SAMSUNG
System: Wear OS
Paired Number: 123456789
Serial Number: SWA-2026-001
Battery Level: 60%
WiFi: Not connected
Bluetooth: Not connected
Tracked Activities: 0
Notifications: 0

Samsung smartwatch charging via magnetic charger.
Charging... Battery is now at 75%.
Samsung smartwatch connected to WiFi.
Samsung smartwatch connected to Bluetooth.
Workout started: Running
Workout stopped.
Notification received: Meeting at 14:00

Watch Info:
```

```
Producer: SAMSUNG
System: Wear OS
Paired Number: 123456789
Serial Number: SWA-2026-001
Battery Level: 75%
WiFi: Connected
Bluetooth: Connected
Tracked Activities: 1
Notifications: 1
```

In the tester class:

- create a list of smartwatches,
- add both Android and Apple watches,
- test all implemented methods,
- print the objects before and after selected operations.

Additional Requirements

- Use encapsulation properly – fields should not be accessed directly from outside the class.
- Use constructors responsibly and avoid code duplication.
- Make sure that battery level never goes below 0% or above 100%.
- Store every completed workout in the activity history.
- Store every received notification in the notification history.

Extra Task

Extend the project by adding timestamps with `LocalDateTime`. Each notification and each completed workout should include the date and time when it was recorded.
Example:

```
[2026-03-21T09:45] Notification: Low battery
[2026-03-21T10:15] Workout completed: Cycling
```